Université du Luxembourg

Introduction to Message Passing Interface (MPI)

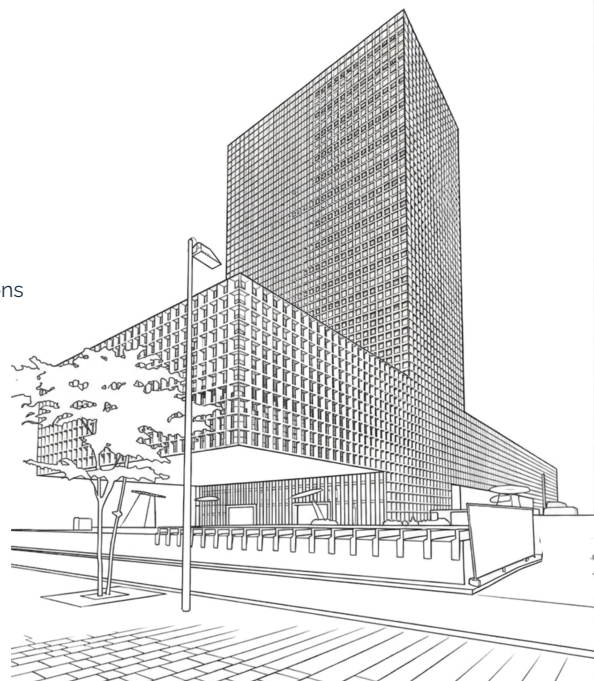
Lesson 9 - Topology, Custom Data types aand Operations

Matteo Barborini

University of Luxembourg - HPC Platform

matteo.barborini@uni.lu

11th of November 2026



Université du Luxembourg

Fourier's heat equation



The time evolution of temperature

Fourier's heat equation

Fourier's heat equation describes the time evolution of the temperature of a system. In 1D it is written as

$$\frac{du(x,t)}{dt} = \frac{\Lambda}{\rho c}\frac{\partial^2}{\partial x^2}u(x,t) \tag{1}$$

where $u(x,t)$ is the local temperature and t is time.

- ρ is the density of the material;
- c is the Specific heat capacity (The amount of heat energy required to change the temperature by 1° per unit of mass);
- Λ is the thermal conductibility (the rate at which heat passes through a given cross-sectional area of a material when driven by a temperature gradient).

Defining the diffusion constant $D = \frac{\Lambda}{\rho c}$ the equation is most commonly rewritten as

$$\frac{\partial}{\partial t}u(x,t) = D\frac{\partial^2}{\partial x^2}u(x,t) \tag{2}$$

Euler discretization

Fourier's heat equation

Defining the diffusion equation

$$\frac{\partial}{\partial t}u(x,t) = D\frac{\partial^2}{\partial x^2}u(x,t) \tag{3}$$

can be discretized on a grid computing the gradients and Laplacians through finite differences introducing a time step Δt and a spacial step Δx:

$$\frac{u(x_m,t_{n+1}) - u(x_m,t_n)}{\Delta t} = D\frac{u(x_{m+1},t_n) - 2u(x_m,t_n) + u(x_{m-1},t_n)}{\Delta x^2}. \tag{4}$$

From this equation we can obtain the time evolution of the solution at a chosen position x_m as

$$u(x_m,t_{n+1}) = u(x_m,t_n) + \frac{D\Delta t}{\Delta x^2}\left[u(x_{m+1},t_n) - 2u(x_m,t_n) + u(x_{m-1},t_n)\right]. \tag{5}$$

It is important to remember that this algorithm is stable only if $\Delta t \leq \frac{\Delta x^2}{2D}$. Example of this algorithm can be found in `9.8.SERIAL_1D_Heat.c`.

Université du Luxembourg

MPI Topology



MPI Topologies

MPI Topology

Default MPI Grouping: Linear rank order $(0, 1, \ldots, N-1)$ often does not match physical hardware or grid-based problem domains.

Grid-based Domains: 2D/3D numerical simulations (e.g., CFD, stencil computation, structural mechanics) require nearest-neighbor communication.

Virtual Topologies:

- Provide a logical mapping between MPI processes and a spatial grid structure.
- Simplify index translation (Rank $\leftrightarrow$ Grid Coordinates).
- Allow the MPI implementation to optimize placement on physical hardware topology.

MPI supports two primary virtual topology abstractions:

Cartesian Topologies

- Regular N-dimensional grids (1D, 2D, 3D, ...).
- Optional periodicity (wrap-around boundary conditions).
- Created using `MPI_Cart_create`.

Graph Topologies

- General, arbitrary network graphs.
- Process connections described by nodes and edges.
- Created using `MPI_Graph_create`.

MPI_Cart_create

MPI Topology

The function `MPI_Cart_create` is used to create a new communicator with Cartesian topology attached.

```
int MPI_Cart_create(
    MPI_Comm old_comm,      // Initial communicator (MPI_COMM_WORLD)
    int num_of_dimensions, // Number of dimensions of Cartesian grid (1D, 2D ...)
    const int dims[],      // Integer array size num_of_dimensions (MPI processes per dim)
    const int periods[],   // Logical array size num_of_dimensions
    int reorder,           // 1 = allow rank reordering, 0 = keep ranks
    MPI_Comm *comm_cart    // Output communicator with topology
);
```

In particular we have that

- **dims**: Specifies the number of MPI processes that partition a given direction.
- **periods**: Specifies wrap-around behavior per dimension (For heat equation non-periodic).
- **reorder**: Allows MPI to reassign ranks to optimize hardware locality.

Finding Neighbors with `MPI_Cart_shift`

MPI Topology

`MPI_Cart_shift` finds the source and destination process ranks for shift operations along a specific dimension.

```
int MPI_Cart_shift(
    MPI_Comm comm,      // Cartesian communicator
    int direction,      // Dimension along which shift occurs (0, 1, ...)
    int disp,           // Displacement (>0 = forward, <0 = backward)
    int *rank_source,   // Rank of process from which caller receives
    int *rank_dest      // Rank of process to which caller sends
);
```

If a boundary is non-periodic, shifted-out ranks return `MPI_PROC_NULL`.

`MPI_PROC_NULL` safely acts as a no-op in MPI point-to-point calls.

Example of 2D Mesh Setup in C

MPI Topology

We create a 2D mesh split into 9 MPI ranks (each dimension is split in 3) can be found in `9.10.MPI_Cart_simple.c`

```c
// First we partition the system
int ranks_per_dims[2] = {3, 4};    // 3x4 grid (12 processes)
int periods[2] = {0, 0};           // Non-periodic boundaries
int reorder = 1;

// Then we Create 2D Cartesian Communicator
MPI_Comm MPI_COMM_2D;
MPI_Cart_create(MPI_COMM_WORLD, 2, ranks_per_dims, periods, reorder, &MPI_COMM_2D);
```

afterwards we interrogate the communicator about the coordinate of a specific rank:

```c
// The name of the rank
MPI_Comm_rank(MPI_COMM_2D, &mpi_rank);
// Find the coordinates of a give rank (int coords[2])
MPI_Cart_coords(MPI_COMM_2D, mpi_rank, 2, coords);

// Find neighbors along dimension 1 (columns) 0 (rows)
MPI_Cart_shift(MPI_COMM_2D, 0, 1, &left, &right);
MPI_Cart_shift(MPI_COMM_2D, 1, 1, &down, &up);
```

Here `MPI_Cart_shift` is used to retrieve the information of the rank numbers of the surrounding neighbors.

Mapping Between Ranks and Coordinates

MPI Topology

Once the Cartesian communicator is created, two functions can convert information between coordinate and rank, such as `MPI_Cart_coords` used in the previous simple example

```
int MPI_Cart_coords(
    MPI_Comm comm,
    int rank,               // Rank of the desired process
    int num_of_dimension,   // Dimensions of the partition
    int coords[]);          // Vector with num_of_dimension elements
```

Another useful function that translates Cartesian grid coordinates into a process rank is

```
int MPI_Cart_rank(
    MPI_Comm comm,
    const int coords[],     // Coordinate vector used to interrogate the env.
    int *rank);
```

In the previous example we have used the first function as

```
// Find the coordinates of a give rank (int coords[2])
MPI_Cart_coords(MPI_COMM_2D, mpi_rank, 2, coords);
```

Going Back to the simple example

MPI Topology

By running the simple example `9.10.MPI_Cart_simple.c` we have the output

```
Rank 0 at (0, 0) -> Left: -2, Right: 4, Down: -2, Up: 1
Rank 1 at (0, 1) -> Left: -2, Right: 5, Down: 0, Up: 2
Rank 3 at (0, 3) -> Left: -2, Right: 7, Down: 2, Up: -2
Rank 2 at (0, 2) -> Left: -2, Right: 6, Down: 1, Up: 3

Rank 4 at (1, 0) -> Left: 0, Right: 8, Down: -2, Up: 5
Rank 5 at (1, 1) -> Left: 1, Right: 9, Down: 4, Up: 6
Rank 6 at (1, 2) -> Left: 2, Right: 10, Down: 5, Up: 7
Rank 7 at (1, 3) -> Left: 3, Right: 11, Down: 6, Up: -2

Rank 9 at (2, 1) -> Left: 5, Right: -2, Down: 8, Up: 10
Rank 8 at (2, 0) -> Left: 4, Right: -2, Down: -2, Up: 9
Rank 10 at (2, 2) -> Left: 6, Right: -2, Down: 9, Up: 11
Rank 11 at (2, 3) -> Left: 7, Right: -2, Down: 10, Up: -2
```

that gives us the mapping created through the MPI_Cart_create function that is visualized in the plot on the right

3 (0,3)	7 (1,3)	11 (2,3)
2 (0,2)	6 (1,2)	10 (2,2)
1 (0,1)	5 (1,1)	9 (2,1)
0 (0,0)	4 (1,0)	8 (2,0)

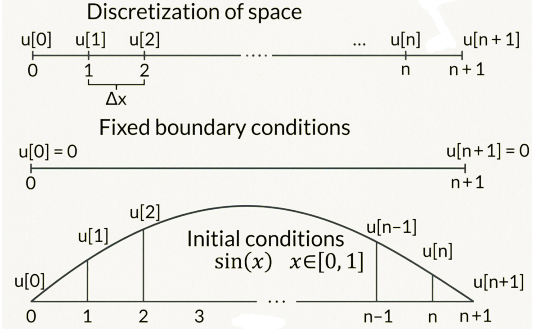
Domain decomposition for the 1D Heat Equation

MPI Topology

Let us assume we want to describe the evolution of temperature on a 1D segment of length L.
The equation that describe its time evolution will be

$$u(x_m, t_{n+1}) = u(x_m, t_n) + \frac{D\Delta t}{\Delta x^2}\left[u(x_{m+1}, t_n) - 2u(x_m, t_n) + u(x_{m-1}, t_n)\right]. \tag{6}$$

where we divide $\Delta x = L/N$ and impose that $\Delta t \leq \frac{\Delta x^2}{2D}$.



Domain decomposition for the 1D Heat Equation

MPI Topology

Each grid point i depends only on its left neighbor ($i-1$) and right neighbor ($i+1$). Subdividing a 1D domain across MPI processes requires exchanging single halo values (ghost cells) with adjacent processes at each time step.

- Interior elements ($1 \ldots N$) updated locally.
- Ghost cells (0 and $N+1$) store data received from neighbors.
- Neighbor ranks determined via 1D `MPI_Cart_shift` (Direction 0: Left / Right)

In the example `9.9.MPI_1D_Heat.c` we first create the Cartesian communication

```
1     // Setup 1D Cartesian Topology
2     int dims[1] = {num_of_ranks};
3     int periods[1] = {0}; // Non-periodic (Dirichlet boundaries: u = 0 at ends)
4     int reorder = 1;
5     MPI_Comm MPI_COMM_1D;
6     MPI_Cart_create(MPI_COMM_WORLD, 1, dims, periods, reorder, &MPI_COMM_1D);
7
8     // Find left and right neighbors for each rank
9     int left, right;
10    MPI_Cart_shift(MPI_COMM_1D, 0, 1, &left, &right);
```

After this the mapping is set.

MPI evolution of the 1D heat equation

MPI Topology

For each time step we repeat the operations:

```
// Send left boundary to LEFT neighbor, receive into right ghost cell from RIGHT neighbor
MPI_Sendrecv(&u[1], 1, MPI_DOUBLE, left, 0,
             &u[num_local + 1], 1, MPI_DOUBLE, right, 0,
             MPI_COMM_1D, MPI_STATUS_IGNORE);

// Send right boundary to RIGHT neighbor, receive into left ghost cell from LEFT neighbor
MPI_Sendrecv(&u[num_local], 1, MPI_DOUBLE, right, 1,
             &u[0], 1, MPI_DOUBLE, left, 1,
             MPI_COMM_1D, MPI_STATUS_IGNORE);

// Computing local Finite Difference Update
for (int i = 1; i <= num_local; i++) {
    u_new[i] = u[i] + r * (u[i - 1] - 2.0 * u[i] + u[i + 1]);
}
```

notice that each process works only on its local block, using the boundary conditions that are shared before the computation of the update.

MPI_Sendrecv combines a send and a receive into a single operation.

```
int MPI_Sendrecv(
            const void *sendbuf, int sendcount, MPI_Datatype sendtype,
                  int dest, int sendtag,
                  void *recvbuf, int recvcount, MPI_Datatype recvtype,
                  int source, int recvtag,
                  MPI_Comm comm, MPI_Status * status)
```

Université du Luxembourg

MPI Derived Types



Motivation for MPI Derived Types

MPI Derived Types

Why do we need derived datatypes?

- **Zero-Copy Transfers:** Avoid explicit manual buffer packing/unpacking in memory.
- **Non-Contiguous Layouts:** Send matrix columns, sub-grids, or strided blocks directly.
- **Heterogeneous Structs:** Pass `struct` types containing mixed primitives safely across architectures.

In MPI, **a datatype is an object consisting of a sequence of the basic datatypes and displacements (in bytes)**.
A datatype can be represented as a sequence of pairs of basic types and displacements (typemap):

$$\text{Typemap} = \left\{(\text{type}_0, \text{disp}_0), (\text{type}_1, \text{disp}_1), \ldots, (\text{type}_{n-1}, \text{disp}_{n-1})\right\} \tag{7}$$

and the type signature

$$\text{Type Signature} = \left\{\text{type}_0, \text{type}_1, \ldots, \text{type}_{n-1}\right\} \tag{8}$$

controls how data items are interpreted when data is sent or received.

```
1 int MPI_Type_get_extent(MPI_Datatype datatype, MPI_Aint *lower_bound, MPI_Aint *extent);
2
3 int MPI_Type_size(MPI_Datatype datatype, int *size)
```

Where `MPI_Aint` corresponds to `long int` in C.

Motivation for MPI Derived Types

MPI Derived Types

In example `9.3.MPI_Datatypes1.c` the functions are run for `MPI_DOUBLE`

```
    long int extent = 0;
    long int lower_bound = 0;
    int size = 0;

    MPI_Type_get_extent(MPI_DOUBLE, &lower_bound, &extent);
    MPI_Type_size(MPI_DOUBLE, &size);

    printf("For MPI_DOUBLE we have\n");
    printf("Lower bound : %ld\n", lower_bound);
    printf("Extent      : %ld\n", extent);
    printf("Size        : %d\n", size);
```

for which we obtain the output

```
For MPI_DOUBLE we have
Lower bound : 0
Extent      : 8
Size        : 8
```

Thus **`lower_bound`** corresopnds to the first element of the type, **`extent`** is the number of bytes of the type (closer to the stride of a datatype: the distance (in bytes) to skip (stride) from the start of one instance of the datatype to the start of another instance of a datatype) and **`size`** is the total size of the variable.

Functions to construct MPI Derived Types

MPI Derived Types

The basic functions to construct costum datatypes are:

Contiguous: produces a new datatype by making a number of copies of an existing one, with the displacements incremented by the extent of the oldtype.

Vector: allows for regular gaps in the displacements. Elements are separated by multiples of the extent of the input datatype.

Hvector(deprecated): same as vector, but elements are separated by a specified number of bytes.

Indexed: In this datatype, an array of displacements of the input datatype is provided; the displacements are measured in terms of the extent of the input datatype.

Hindexed (deprecated): This is like indexed, but the displacements are measured in bytes.

Struct: This provides a fully general description. In fact, if the input arguments consist of the basic MPI datatypes, the input is just the typemap.

MPI_Type_contiguous

MPI Derived Types

The MPI_Type_contiguous function produces a new datatype of repeated entries (count) of an existing one

```
int MPI_Type_contiguous(
    int count,
    MPI_Datatype *oldtype,
    MPI_Datatype *newtype)
```

count: The number of copies to create of datatype (oldtype).

oldtype: Datatype of the elements that we need to repeat (e.g., MPI_INT, MPI_DOUBLE).

newtype: Name of the newtype to handed the vector datatype that we have created

An example of the usage of this function can be found in `9.4.MPI_Type_contiguous.c`. After the function MPI_Type_contiguous is called and the new datatype is allocated one must use

```
int MPI_Type_commit(MPI_Datatype *datatype)
```

that tells the library to optimize the datatypes internal representation for fast memory packing and unpacking during communication. Before finalizing the MPI environment we must call

```
int MPI_Type_free(MPI_Datatype *datatype)
```

which is used to deallocate a derived datatype.

MPI_Type_vector

MPI Derived Types

The MPI_Type_vector function is similar to the MPI_Type_contiguous type, but it allows for regular gaps in the displacements between elements

```
int MPI_Type_vector(
    int count,
    int blocklength,
    int stride,
    MPI_Datatype oldtype,
    MPI_Datatype *newtype
);
```

count: The number of blocks to create.

blocklength: The number of elements in each individual block (they must all be the same size).

stride: The distance (measured as a multiple of oldtype extents) from the start of one block to the start of the next block.

oldtype: The underlying datatype of the elements (e.g., MPI_INT, MPI_DOUBLE).

newtype: The handle for the newly created vector datatype.

After the definition of the datatype we still need to commit it calling the MPI_Type_commit function and destroy it through the MPI_Type_free function before finalizing the MPI environment.

MPI_Type_indexed

MPI Derived Types

In example `9.6.MPI_Type_indexed.c` we use the function `MPI_Type_indexed` to create an MPI_Datatype used to communicate the upper triangular part of a matrix

```
int MPI_Type_indexed(
    int count,
    const int blocklengths[],
    const int displacements[],
    MPI_Datatype oldtype,
    MPI_Datatype *newtype
);
```

count: The number of blocks (which corresponds to the number of fields or entries in your structural arrays).

array_of_blocklengths[]: An array specifying how many elements are in each block. (Usually 1 for standard scalar struct members, but can be higher if a member is an array).

array_of_displacements[]: An array of absolute byte displacements (expressed as MPI_Aint) for each block from the beginning of the buffer.

array_of_types[]: An array of MPI_Datatype handles corresponding to each block (e.g., MPI_INT, MPI_DOUBLE, MPI_CHAR).

newtype: The newly created struct datatype.

MPI_Type_create_struct

MPI Derived Types

In example `9.7.MPI_Type_struct.c` we use the function `MPI_Type_create_struct` to create an MPI_Datatype used to communicate a **particle** structure.

```
int MPI_Type_create_struct(
    int count,                                  // Number of variables in the struct
    int array_of_blocklengths[],                // Number of elements for each variable
    const MPI_Aint array_of_displacements[],    // Addresses of the variable
    const MPI_Datatype array_of_types[],        // Array of types for each variable in struct
    MPI_Datatype *newtype                       // User defined MPI_Datatype name
);
```

To create a new derived datatype by modifying the lower bound and extent of an existing datatype one can use the function

```
int MPI_Type_create_resized(
    MPI_Datatype oldtype,                       // Old datatype
    MPI_Aint lb,                                // Lower bound
    MPI_Aint extent,                            // This is the sizeof(oldtype)
    MPI_Datatype *newtype                       // User defined MPI_Datatype name
);
```

Using this function is the standard way to account for structural padding when sending arrays of custom C structures or non-contiguous data blocks in MPI.

Université du Luxembourg

MPI Customized Operations



Reduce and Scan operations in MPI

MPI Customized Operations

Reduce and Allreduce functions are used to perform a reduction operation (like sum, max, min, etc.) across all processes in a communicator.

```
int MPI_Reduce [MPI_Ireduce] (
    const void *data_to_reduce,
    void *data_reduced,
    int data_size,
    MPI_Datatype data_type,
    MPI_Op reduce_operation,
    int receiving_rank,
    MPI_Comm comm,
    [MPI_Request *request] )
```

```
int MPI_Allreduce [MPI_Iallreduce] (
    const void *data_to_reduce,
    void *data_reduced,
    int data_size,
    MPI_Datatype data_type,
    MPI_Op reduce_operation,
    MPI_Comm comm,
    [MPI_Request *request] )
```

Scan and Exscan functions are used to perform a prefix reduction operation across all processes in a communicator.

```
int MPI_Scan [MPI_Iscan] (
   const void *send_buffer,
   const void *receive_buffer,
   int size_of_data,
   MPI_Datatype data_type,
   MPI_Op reduction_operation,
   MPI_Comm comm,
   [MPI_Request *request] )
```

```
int MPI_Exscan [MPI_Iexscan] (
   const void *send_buffer,
   const void *receive_buffer,
   int size_of_data,
   MPI_Datatype data_type,
   MPI_Op reduction_operation,
   MPI_Comm comm,
   [MPI_Request *request] )
```

Partial list of MPI standard operations

MPI Customized Operations

A partial list of the available reduction operations in MPI is shown in the following table.

MPI_Op	MPI_Datatype	Description
MPI_MAX	Integer, Floating-point	Maximum value
MPI_MIN	Integer, Floating-point	Minimum value
MPI_SUM	Integer, Floating-point, (Complex)	Sum
MPI_PROD	Integer, Floating-point, (Complex)	Product
MPI_LAND	Integer, Logical	Logical AND
MPI_BAND	Integer, Byte	Bitwise AND
MPI_LOR	Integer, Logical	Logical OR
MPI_BOR	Integer, Byte	Bitwise OR
MPI_LXOR	Integer, Logical	Logical XOR
MPI_BXOR	Integer, Byte	Bitwise XOR
MPI_MAXLOC	Structs (Value + Index)	Maximum value and its associated index/rank
MPI_MINLOC	Structs (Value + Index)	Minimum value and its associated index/rank
MPI_REPLACE	Any	Overwrites target with initial value (primarily used in RMA/one-sided communication)

Customize `MPI_Op` 1

MPI Customized Operations

While **standard MPI Operators** provide built-in operators like `MPI_SUM`, `MPI_MAX`, and `MPI_PROD`, acting on standard datatypes, **custom MPI Operators** allow users to define their own reduction operations for complex data structures or non-standard combining rules.

The costume function must have this exact structure:

```
typedef int User_Defined_function(
    void *invec,
    void *inoutvec,
    int *len,
    MPI_Datatype *datatype)
```

where:

- `invec`: Input vector (values from other processes).
- `inoutvec`: Input/Output vector (accumulated result; modified in place).
- `len`: Number of elements of type `datatype`.
- `datatype`: The datatype argument is a handle to the data type that was passed into the call.

Once the function is globally defined to create the MPI operation we must use the following MPI function:

```
int MPI_Op_create(MPI_User_function *User_Defined_function,
    int commute,
    MPI_Op *operation)
```

Customize `MPI_Op` 2

MPI Customized Operations

The user defined function (`User_Defined_function`):

Is the one previously defined by the user, which will be called during the reduction operation. It must adhere to the specified signature and handle the data types correctly.

The Commutativity Flag (`commute`):

- Pass 1 (true) if $A \oplus B = B \oplus A$, thus order does not matter (allows MPI to optimize tree reduction routing).
- Pass 0 (false) if $A \oplus B \neq B \oplus A$, thus order matters (e.g., matrix multiplication or string concatenation).

The presence of the commutative flag allows an MPI implementation more freedom in determining the order in which it computes the result.

The output (`MPI_Op *operation`):

The output argument is the new operation (in C of type MPI_Op, Fortran has a different structure ...) that can be passed to routines such as MPI_Reduce, MPI_Scan, etc.

Before closing the MPI environment, the user must free the operation using:

```
1 int MPI_Op_free(MPI_Op *operation)
```

Example of `MPI_Op` usage 1

MPI Customized Operations

In the example `9.1.MPI_Op_create.c` the custom operation is defined for the MPI_DOUBLE datatype, but it can be easily extended to other datatypes. The custom operation is a saturating addition, which caps the sum at 100.

```c
void saturated_sum_double(void *invec, void *inoutvec, int *len, MPI_Datatype *datatype)
{
   if (*datatype == MPI_DOUBLE)
   {
      double *in = (double *)invec;
      double *inout = (double *)inoutvec;
      double sum = 0.0;
      for (int i = 0; i < *len; i++)
      {
         sum = in[i] + inout[i];
         inout[i] = (sum > 100) ? 100 : sum;
      }
   }
   else
   {
      fprintf(stderr, "ERROR!!!: Unsupported datatype for saturated_double_add.\n");
      MPI_Abort(MPI_COMM_WORLD, 1);
   }
}
```

the operation is then defined in the main by calling

```c
    MPI_Op MPI_SUM_CAP100;
    MPI_Op_create(*saturated_sum_double, 1, &MPI_SUM_CAP100);
```

notice that at the end of the program we free the vector of the custom sum MPI_SUM_CAP100

```c
    MPI_Op_free(&MPI_SUM_CAP100);
```

Example of `MPI_Op` usage 2

MPI Customized Operations

In the more copmlex example `9.2.MPI_Op_create_polymorphic.c` the custom operation is defined for different MPI_Datatypes.

```
1  void saturated_sum(void *invec, void *inoutvec, int *len, MPI_Datatype *datatype)
2  {
3     if (*datatype == MPI_DOUBLE)
4     {
5        double *in = (double *)invec;
6        double *inout = (double *)inoutvec;
7        double sum = 0.0;
8        for (int i = 0; i < *len; i++)
9        {
10          sum = in[i] + inout[i];
11          inout[i] = (sum > 100) ? 100 : sum;
12       }
13    }
14    else if (*datatype == MPI_INT)
15    {
16       int *in = (int *)invec;
17       int *inout = (int *)inoutvec;
18       int sum = 0;
19       for (int i = 0; i < *len; i++)
20       {
21          sum = in[i] + inout[i];
22          inout[i] = (sum > 100) ? 100 : sum;
23       }
24    }
25 ...
26    else
27    {
28       fprintf(stderr, "ERROR!!!: Unsupported datatype for saturated_add.\n");
29       MPI_Abort(MPI_COMM_WORLD, 1);
30    }
31 }
```